Modular main()

Document:	P1203R0
Audience:	EWG

Authors:	Boris Kolpackov (Code Synthesis), Richard Smith (Google)
Reply-To:	boris@codesynthesis.com, richard@metafoo.co.uk

Date:	2018-10-05

Contents

1 Abstract

This paper discusses the benefits of being able to define `main()` in a module and proposes to allow both exported and non-exported variants with slightly different semantics.

2 Background

Per the discussion at the Bellevue meeting, there appears to be agreement that modularizing a codebase should eventually result in the replacement of all non-

modular translation units with modules. This, however, brings the question of what happens to the translation unit that defines `main()`.

While it will be possible to define `main()` in the global module fragment, this will be both cumbersome and without any clear benefits other than making the translation unit itself a module. Specifically, due to the restrictions placed on the global module fragment, such a `main()` would have to be defined in a separate file that is `#include`d into the module translation unit. And such a definition wouldn't have visibility of the module's names, for example, implementation details that would be naturally placed into the module.

The question is then whether it could be useful to place `main()` into a module and, if so, with what semantics? Placing a function into a module's purview affects two orthogonal aspects: *name visibility* and *ownership*. Specifically, the function now has visibility of all the names declared in a module and the function is now owned by that module. Let's discuss whether and how each could be useful in the context of `main()`.

Name visibility would allow "hiding" `main()`'s implementation details (functions, classes) in the same module in order to avoid symbol conflicts (currently achieved with `static` and/or unnamed namespaces). For example:

```
export module main;

class application
{
  ...
};

export int main ()
{
  application a;
  a.run ();
}
```

The ability to "see" a module's non-exported names would also be required for a module's unit testing. [Note: Without modular `main()` the best option is probably

to provide an (appropriately decorated, for example with the module name) exported test function that is then called by the traditional `main()`.]

At first glance, module ownership for `main()` might appear counterproductive; after all, we should have a single `main()` in a program. However, consider module unit testing as an example: a module may wish to provide `main()` that implements its unit tests. And a library may contain multiple such modules, which means it may contain multiple `main()`s. In this context, having `main()` owned by a module would avoid any conflicts.

> The ability to have a module-specific `main()` would allow combining the module implementation and its unit tests in a single translation unit, an arrangement advocated by many modern programming languages.

For example:

```
export module util.hashmap;

export class hashmap
{
  ...
};

int main ()
{
  // Unit tests for hashmap.
}
```

3 Proposal

3.1 Basic Functionality

We propose to allow defining `main()` in a module as an *exported* function. Such a definition has visibility of the module's names but whether it is owned by the

module is implementation-defined (that is, it may still be `extern "C"` and belong to the global module). All existing semantics and restrictions, such as that a program shall have a single `main()` function, still apply.

3.2 Extended Functionality

We further propose to allow defining a *non-exported* `main()` that, in addition to having the module's name visibility, would also be owned by the module. Unlike the exported `main()`, a program may contain multiple such functions with one of them selected using an implementation-defined mechanism. [Note: For example, a link-time switch that specifies the module to take the `main()` from.]

> If a program contains both one or more exported `main()`s and a non-exported/non-modular variant, then the latter is selected by default.

> We realize that supporting this extended functionality may require additional mechanisms in the underlying linking technology (for example, symbol aliasing/redirection). Note, however, that as a last resort an implementation can treat a non-exported `main()` as an ordinary function and generate a *thunk* `main()` that calls it.